

UERJ – INSTITUTO DE QUÍMICA
PROGRAMA DE PÓS-GRADUAÇÃO EM ENGENHARIA QUÍMICA
TÓPICOS ESPECIAIS EM ENG. QUÍMICA: APRENDIZADO DE MÁQUINA

Capítulo 9: Modelagem em Aprendizado de Máquina

Lista de Exercícios: Use a linguagem R ou Python (livre escolha)

1ª. Questão: Valor 10,0 pontos

Use o conjunto de dados Steel Plates Faults, disponível no repositório do UCI Machine Learning. O conjunto reúne medições automáticas realizadas na superfície de placas de aço inoxidável, com o objetivo de classificar o tipo de falha detectada — tarefa análoga à detecção e ao diagnóstico de falhas em processos industriais.

Página do conjunto de dados: [Steel Plates Faults — UCI](#)

Download direto (arquivo .zip): [steel+plates+faults.zip](#)

Estrutura do arquivo

O download traz dois arquivos. O Faults.NNA contém os dados, separados por tabulação e sem linha de cabeçalho. O Faults27x7_var traz, um por linha, os nomes das 34 colunas. Atribua os nomes manualmente no momento da importação.

As 27 primeiras colunas são os preditores, todos contínuos, entre os quais medidas geométricas da região com defeito (X_Minimum, Y_Maximum, Pixels_Areas, X_Perimeter), medidas de luminosidade (Sum_of_Luminosity, Minimum_of_Luminosity), características da placa (Length_of_Conveyer, Steel_Plate_Thickness, TypeOfSteel_A300, TypeOfSteel_A400) e índices derivados (Edges_Index, Square_Index, LogOfAreas, SigmoidOfAreas, entre outros).

As 7 últimas colunas — Pastry, Z_Scratch, K_Scratch, Stains, Dirtiness, Bumps e Other_Faults — codificam o tipo de falha em formato binário mutuamente exclusivo: cada registro pertence a exatamente uma classe.

O conjunto contém 1.941 registros e não apresenta valores ausentes.

Preparação dos dados

Antes de iniciar a modelagem, execute os procedimentos abaixo e descreva-os em sua resposta:

- Reúna as sete colunas binárias em uma única variável categórica de resposta, com sete níveis. Confirme que a soma dessas sete colunas é igual a 1 em todos os registros.
 - As classes são fortemente desbalanceadas: a mais frequente (Other_Faults) reúne 673 registros e a menos frequente (Dirtiness), apenas 55 — razão aproximada de 12 : 1.
 - Os preditores estão em escalas muito distintas: a razão entre a maior e a menor variância é da ordem de 10^{14} . Avalie a necessidade de padronizá-los, considerando os algoritmos a serem empregados, e justifique a decisão.
-
- a) Verifique a presença de valores faltantes e examine a redundância entre os preditores. Observe, em particular, as variáveis TypeOfSteel_A300 e TypeOfSteel_A400. (1,0 ponto)
 - b) Proceda ao pré-processamento e à transformação dos dados, justificando as escolhas feitas. Identifique os pares de preditores com correlação absoluta superior a 0,90 e decida como tratá-los. (1,5 ponto)
 - c) Divida o conjunto em treinamento (80%) e teste (20%), preservando a proporção das classes. Justifique a metodologia de partição adotada. (1,0 ponto)
 - d) Considerando que as classes estão desbalanceadas, escolha um algoritmo de downsampling e outro de upsampling para corrigi-las. Aplique-os exclusivamente ao conjunto de treinamento e explique por que o conjunto de teste não deve ser reamostrado. (1,5 ponto)
 - e) Analise o efeito dos algoritmos empregados no balanceamento das classes em relação ao conjunto de dados original. Informe quantos registros restam no conjunto de treinamento em cada cenário. (1,0 ponto)
 - f) Compare o desempenho dos algoritmos C5.0 e Random Forest usando os parâmetros de mérito, nos três cenários: dados originais, com downsampling e com upsampling. (2,5 pontos)
 - g) Qual é o algoritmo que apresentou o melhor desempenho na tarefa de classificação? Justifique sua escolha, considerando que a exatidão global é

pouco informativa em problemas desbalanceados, e indique quais métricas foram adotadas. (1,5 ponto)

Observação: o algoritmo C5.0 é implementado no pacote C50 do R e não possui equivalente nativo no scikit-learn. Em Python, utilize uma árvore de decisão com boosting — por exemplo, AdaBoost ou Gradient Boosting sobre DecisionTreeClassifier — e indique, em sua resposta, a implementação adotada. Para o balanceamento de classes, os pacotes caret e themis (R) e a biblioteca imbalanced-learn (Python) oferecem as funções necessárias.